ICREFINE: Diagnosing the Limits of Iterative Cheatsheet Transfer Across Model Families

Anonymous Author(s)
Affiliation
Address
email

1 Introduction

In-context learning (ICL) achieves strong performance on many NLP tasks by prepending demonstrations to the inference prompt, yet the quality of those demonstrations depends heavily on selection, ordering, and format. Recent work has explored a more compact alternative: replacing a large demonstration bank with a *cheatsheet*: a concise, structured summary of task heuristics that is reusable across all test inputs [Honda et al., 2025]. A single-pass cheatsheet generated from training examples provides a competitive baseline at a fraction of the per-input token cost of many-shot ICL, and its fixed format makes it straightforward to deploy across model families at inference time.

A natural next question is whether the quality of a cheatsheet can be improved by *iterative refinement*: cycling through training failures, patching the rules, and extending the cheatsheet with structured corrective examples. Iterative refinement has proven effective in prompt optimization more broadly [Yang et al., 2024, Zhou et al., 2023, 2022], and there is reason to think that a small train model, if given explicit feedback loops, might produce cheatsheets competitive with those generated in a single pass by a more capable model. But a critical question stands apart: does a cheatsheet refined against one model *transfer* when applied to a different, stronger model at inference time? Transfer matters because in practice the train model is chosen for efficiency (cheap API calls during refinement) while the inference model is chosen for quality. If iterative refinement encodes corrections that are idiosyncratic to the train model's failure modes, the resulting cheatsheet may be actively harmful to other models.

We introduce **ICREFINE**, a three-phase iterative cheatsheet refinement pipeline. **Phase 0** bootstraps a prior-knowledge (PK) ruleset from 75 training examples in a single pass. **Phase 1** iteratively patches the PK through a sample-and-accept loop that scores failures, generates candidate revisions, and applies fix-rate and regression gates. **Phase 2** partitions remaining training failures by structural error type and generates one structured case study per coherent failure cluster, using a novel `ACTIVATE IF / WHY` format that specifies trigger conditions and correction rationale. We evaluate ICREFINE (train model: gpt-4.1-mini) on five non-ceiling tasks from BIG-Bench Hard [Suzgun et al., 2022], transferring the resulting cheatsheets to four stronger evaluation models: GPT-4.1, Claude-3.7-sonnet, Gemini-2.0-flash, and Llama-3.3-70b. The baseline is the static Cheatsheet-ICL (CS-ICL) approach of Honda et al. [2025], generated by gpt-4.1 (not mini) in a single pass.

What we did not anticipate is that the full ICREFINE pipeline *underperforms* CS-ICL on *all four* non-train model families in point estimate (-1.0 to -3.7 pp), with the deficit statistically distinguishable from zero for GPT-4.1 and Claude. This failure is not uniform noise; it is *diagnosable*. We use this failure as a diagnostic instrument: by identifying three mechanistically distinct failure modes, confirming each with targeted interventions, and showing that the pattern replicates across training model families, we establish that the transfer gap is a *property of task structure* rather than of any correctable pipeline design choice. Understanding why iterative refinement fails to transfer turns out to be more informative than a positive result would have been.

Contributions.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

1. We propose ICREFINE, a three-phase pipeline that combines iterative prior-knowledge patching (Phase 1) with partition-based case study generation in a structured `ACTIVATE IF / WHY` format (Phase 2).
2. We present a systematic evaluation of cheatsheet transfer across five model families, demonstrating that the transfer gap is *task-determined* rather than model-determined: the same failure pattern replicates with both gpt-4.1-mini and gemini-2.0-flash as the train model.
3. We identify and characterize three failure modes of iterative refinement (oracle contamination, partition overfitting, and bootstrap convergence limits) and show that each is separately diagnosable.
4. We introduce two targeted interventions: an Evolutionary Algorithm Phase 1 that addresses bootstrap convergence ($+6.0$ pp GS PK accuracy), and oracle-free Phase 2 generation that reduces oracle contamination ($+1.9$–$+3.8$ pp CJ accuracy across all models). The combined EA+no-oracle pipeline remains below CS-ICL, and we show that the residual gap is attributable to task structure rather than to any addressable design flaw. This establishes what would need to change (structurally diverse, model-independent failure modes) for iterative cheatsheet refinement to reliably transfer.

2 Preliminaries

Task setting. We consider multi-class classification tasks where each input x is mapped to a label $y \in \mathcal{Y}$ via an LLM prompted with a fixed instruction. We have access to a labeled training set $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}$ and evaluate on a held-out test set. At inference time, no labeled demonstrations are provided in the prompt; the model generates a chain-of-thought and produces a final verdict. All evaluations use *reasoning-first* (RF) scoring: the model writes its reasoning before emitting `VERDICT: (X)`.

Cheatsheet. A *cheatsheet* $\mathcal{C} = (\text{PK}, \mathcal{S})$ consists of two components prepended to every test prompt: (i) a **prior knowledge** (PK) block—a free-form natural-language ruleset ($\leq$12 000 characters) encoding general task heuristics and common error corrections; and (ii) a set of **case studies** $\mathcal{S}$, each in `ACTIVATE IF / WHY` format: a set of structural trigger predicates over the input, a correction rationale, and annotated support examples. A static cheatsheet ($|\mathcal{S}| = 0$) is the CS-ICL baseline of Honda et al. [2025]; ICREFINE produces cheatsheets with both components.

Train and eval models. We distinguish the **train model** M_{tr}, used for all pipeline roles during refinement (scoring, rule-patching, case study generation), from **evaluation models** $\{M_{\text{ev}}\}$, which consume the cheatsheet at inference time. We say a cheatsheet *transfers* if it improves M_{ev} accuracy above the CS-ICL baseline when $M_{\text{ev}} \neq M_{\text{tr}}$. Transfer is non-trivial because corrections derived from M_{tr}'s failure modes may not match M_{ev}'s failure modes, and may actively mislead a more capable model.

3 ICREFINE

3.1 Cheatsheet format

ICREFINE produces a *cheatsheet* consisting of two components, both prepended to the inference prompt. The **prior knowledge (PK)** is a free-form natural-language ruleset ($\leq$12 000 characters) encoding general task heuristics. **Case studies** follow a structured `ACTIVATE IF / WHY` format: each case study specifies a set of trigger conditions (structural predicates over the input), a correction rationale, and annotated support examples (see Listing 1 in Section 6.1 for a concrete example). At inference time, the full cheatsheet is prepended verbatim to each test prompt; the model applies relevant case studies alongside the PK rules when generating its answer.

3.2 Phase 0: bootstrap

The pipeline initialises the PK by prompting the train model with the first $n_{\text{boot}} = 75$ training examples and requesting a concise rules-and-heuristics summary (single pass, no iteration). Phase 0 follows the cheatsheet generation style of Honda et al. [2025], with the train model (gpt-4.1-mini) in place of the full gpt-4.1 used by both their baseline and ours (see Section 4).

3.3 Phase 1: prior-knowledge patching

Phase 1 iteratively revises the PK using a sample-and-accept loop. At each iteration:

1. Score all 100 training items against the current PK; collect failures $\mathcal{F}_t$.
2. Sample $\min(15, |\mathcal{F}_t|)$ failures; optionally append gold chain-of-thought (`item["reason"]`, oracle injection) as a "Correct reasoning" contrast block.
3. Generate one candidate PK revision (temperature 0.3).
4. Apply two acceptance gates in sequence: **(a) fix-rate gate**: re-score the 15 shown failures; accept if fix_rate ≥ 0.20; **(b) regression gate**: sample $\min(40, |\mathcal{C}_t|)$ currently-correct items; accept if regress_rate ≤ 0.20.
5. If both gates pass, replace the PK and re-score the full training set.

Phase 1 exits when: training accuracy ≥ 0.85, two consecutive iterations produce no accepted patch, or 4 iterations are exhausted. The 4-iteration cap is motivated by API cost: on the tasks studied here, most of the Phase 1 gain arrives in the first two accepted patches; iterations 3–4 rarely move accuracy more than 1–2 pp. Relaxing the cap is a direction for future work.

3.4 Phase 2: partition-based case study generation

Phase 2 partitions remaining training failures by structural error type, then generates one case study per non-trivial partition. Partitions are computed deterministically from item-level structural features (no LLM involvement); only partitions with ≥ 3 failures are processed. For each active partition:

1. Generate $N = 3$ candidate case studies in parallel at temperatures $[0.3, 0.6, 0.9]$. Each candidate is conditioned on the partition's failure set, the current cheatsheet, and (optionally) oracle context. Structural `ACTIVATE IF` predicates derived from the partition key are prepended to every candidate to guarantee correct-partition activation.
2. Mini-evaluate all candidates against the partition's failure set; select the highest fix-rate candidate.
3. Apply two acceptance gates: **(a) fix-rate gate**: accept if fix_rate ≥ 0.30; **(b) regression gate**: sample ≤ 50 partition-matched correct items; accept if regression ≤ 0.15.
4. If both pass, add the case study to the cheatsheet; otherwise archive the candidate (up to 8 per partition) for re-evaluation in later iterations.
5. Up to 3 generation attempts are made per partition per outer iteration; archived candidates are re-evaluated before each new attempt.

Phase 2 exits when all partitions retire (≤ 2 failures remaining), two consecutive outer iterations add no case study, or 5 outer iterations are exhausted. When ≥ 3 case studies exist, a similarity gate checks whether new candidates duplicate or can be merged with existing ones before acceptance.

3.5 Evolutionary Algorithm Phase 1

Standard Phase 1 fails when the bootstrap PK already exceeds the 85% accuracy goal, applying zero patches. To overcome this, the **EA Phase 1** variant replaces single-candidate search with population-based exploration.

Population. Three members are maintained with distinct generation roles: *conservative* (temperature 0.2; targeted clarification edits), *generative* (temperature 0.7; unconstrained rewrites), and *structural* (temperature 0.3; explicit TIGHTEN/SPLIT/ADD_GUARD operations). All members are scored on a fixed 80%/20% train/validation split; only validation accuracy drives selection.

Per-generation loop. In generation 1, the failure set is partitioned disjointly across members so each sees a distinct failure slice. From generation 2 onward, each member samples 60% of its own failures, rotated to avoid repetition. Candidates are accepted if they satisfy: **(a) size**: $|\text{candidate}| \leq 12{,}000$ characters; **(b) curriculum net-gain**: $n_{\text{fixed}} \geq \lambda(g) \cdot n_{\text{regressed}}$, where $\lambda(g) = 1.0 + (g-1)/(G-1)$ ramps from 1.0 (loose, generation 1) to 2.0 (strict, final generation); **(c) absolute regression cap**: $n_{\text{regressed}} \leq \max(3,\ 0.20 \cdot |\mathcal{C}_{\text{pool}}|)$. Regression is measured on $\min(20, |\mathcal{C}_{\text{own}}|)$ own-correct items plus $\min(10, |\mathcal{C}_{\text{sib}}|)$ items from each sibling's correct pool.

Survivor selection and crossover. After each generation, the top $k = 2$ members by validation accuracy survive. If ≥ 2 survivors exist, a crossover step prompts the train model to merge them into a child PK that inherits each survivor's unique wins while avoiding their shared failures (temperature 0.3). The child is scored and added to the population; the best-ever PK across all generations is maintained as an elite and replaces the weakest member each generation. The algorithm exits when all members are static for ≥ 2 consecutive generations or 4 generations are exhausted.

3.6 Oracle injection

When available, the gold chain-of-thought (`item["reason"]`) is injected into both Phase 1 and Phase 2 prompts as a "Correct reasoning" contrast block alongside each failure. The injection priority is: task-specific pre-computed oracle > `item["reason"]` dataset field > external oracle CSV lookup.

Oracle injection was a *default design choice* in the initial v3 pipeline, adopted on the assumption that gold reasoning would help the train model identify better correction rules. It was not originally conceived as an experimental knob. After observing that CJ transfer accuracy was substantially below the CS-ICL baseline despite high training accuracy, we hypothesised oracle contamination as the cause: oracle-assisted Phase 1 and Phase 2 prompts may produce rules that encode the train model's oracle-assisted reasoning style rather than model-independent correction patterns. Experiment **E3** tests this hypothesis by disabling oracle injection for both phases (`-no-phase1-oracle -no-phase2-oracle`); Section 6.2 reports the causal effect on transfer accuracy.

4 Experimental Setup

Reasoning-first (RF) scoring. All conditions are evaluated with *reasoning-first* (RF) scoring: the model generates a free-form chain-of-thought, then outputs `VERDICT: (X)` as the final line. RF scoring is motivated by a striking format artifact we observed in earlier CoT evaluations: tasks such as web_of_lies scored 42–66% under verdict-first CoT prompting but 99–100% under RF prompting—a gap of over 30 pp driven entirely by prompt format, not model capability or cheatsheet quality. Because standard CoT evaluation conflates format sensitivity with model performance, we use RF scoring throughout; all comparisons are therefore internally consistent and free of this artifact. Readers evaluating claims in the broader CoT-evaluation literature should note that verdict-first scoring can produce similar format-induced gaps, and results reported under different prompt formats may not be directly comparable. We regard this format sensitivity as a noteworthy finding in its own right—it affects any cheatsheet-ICL evaluation that uses verdict-first CoT scoring—and plan to report it with a systematic comparison table in an appendix of the full submission.

Dataset. We use BIG-Bench Hard (BBH): 11 tasks, 100 training and 87–250 test examples per task. Seven tasks reach $\geq 95\%$ accuracy after Phase 0 bootstrap alone and are excluded from detailed analysis (boolean_expressions, navigate, web_of_lies, logical_deduction_three, sports_understanding, formal_fallacies on the train model, snarks). The five non-ceiling tasks are **geometric_shapes** (GS), **formal_fallacies** (FF), **snarks**, **disambiguation_qa** (DQ), and **causal_judgement** (CJ).

Models. The **train model** for all pipeline roles (scoring, PK patching, case study generation) is gpt-4.1-mini. Transfer is measured on four **non-train evaluation models**: GPT-4.1, Claude-3.7-sonnet, Gemini-2.0-flash, and Llama-3.3-70b-instruct. The **CS-ICL** baseline is a single-pass cheatsheet generated by gpt-4.1 (not mini) from all 100 training examples, matching the approach in Honda et al. [2025]; it contains no case studies and undergoes no iterative refinement.

Seed variance. API evaluations exhibit backend-routing non-determinism at temperature zero, producing 2–4 pp variance across independent runs. We run the v3, E3, and EA conditions three times each (seeds 1–3) from independent API calls and report 3-seed means throughout; single-run results are noted explicitly in captions.

Table 1: Non-train transfer—average RF accuracy on five non-ceiling tasks (GS, FF, Snarks, DQ, CJ). CS-ICL: static one-shot baseline (gpt-4.1 generated). PK only: ICREFINE prior knowledge, no case studies. Full: complete v3 pipeline. Δ is full minus CS-ICL (pp). 95% CI bootstraps over the 5 tasks (resampling with replacement, 10,000 iterations); * indicates CI excludes zero. **All columns are 3-seed means** (v3 pipeline seeds 1–3; FF averaged over 2 seeds; CS-ICL re-scored 3 times with the same RF prompting used for ICREFINE evaluations).

Model	CS-ICL	PK only	Full	Δ	95% CI
GPT-4.1	87.1%	83.9%	83.7%	-3.4	$[-6.2, -0.7]^*$
Claude-3.7	87.7%	86.4%	84.0%	-3.7	$[-6.6, -0.9]^*$
Gemini-2.0	81.4%	80.6%	80.4%	-1.0	$[-3.5, +0.6]$
Llama-3.3	78.3%	76.8%	75.7%	-2.6	$[-6.8, +1.2]$

5 Results

The answer to the central question is **no**: iterative cheatsheet refinement does not improve on a strong static baseline at transfer time.[1,2] The full ICREFINE pipeline (train model: gpt-4.1-mini) underperforms the static CS-ICL baseline (generated by gpt-4.1 in a single pass) on all four non-train model families, with statistically significant deficits for GPT-4.1 and Claude.

This failure is not opaque. Three mechanistically distinct failure modes explain most of the gap, each mapping to a specific task: oracle contamination in Phase 2 case studies, partition overfitting in prior-knowledge search, and bootstrap convergence limits in Phase 1. The failure pattern is *task-determined*: it replicates when the train model switches from gpt-4.1-mini to gemini-2.0-flash. Two targeted interventions partially mitigate two of the three failure modes; the best-available combined pipeline still falls below CS-ICL for GPT-4.1 and Llama. The gap narrows but does not close.

5.1 Transfer gap: the full pipeline underperforms CS-ICL on non-train models

Table 1 and Figure 1 report average RF accuracy over the five non-ceiling tasks (GS, FF, Snarks, DQ, CJ) on four non-train evaluation models, averaged over three independent pipeline runs (seeds 1–3; see Section 6.6). With CS-ICL re-scored over three independent seeds (Table 1), the full pipeline falls below the 3-seed CS-ICL mean for all four non-train model families in point estimate: GPT-4.1 (-3.4 pp), Claude (-3.7 pp), Llama (-2.6 pp), Gemini (-1.0 pp). A bootstrap CI over the five tasks (resampling with replacement, 10,000 iterations) confirms that **GPT-4.1 and Claude deficits are statistically distinguishable from zero**: GPT-4.1 95% CI $[-6.2, -0.7]$ pp; Claude 95% CI $[-6.6, -0.9]$ pp. Gemini ($[-3.5, +0.6]$) and Llama ($[-6.8, +1.2]$) have CIs that cross zero. The PK-only ablation (prior knowledge without case studies) outperforms the full pipeline for Claude: Claude PK-only (86.4%) exceeds Claude full (84.0%) by $+2.4$ pp, identifying Phase 2 case studies as the primary source of the transfer gap for this model family.

These CIs capture task-selection variance (which 5-task subset drives the average) but not per-example sampling variance. The causal_judgement task drives the majority of the cross-model deficit (Section 6.2): excluding CJ from the 5-task average narrows the gap substantially. Gemini reaches near-zero ($+0.1$ pp; 4-task: 84.5% vs. CS-ICL 84.4%), while Claude (-2.8 pp; 88.6% vs. 91.4%) and GPT-4.1 (-2.1 pp; 87.8% vs. 89.9%) retain reduced but still negative gaps. The negative result is therefore *concentrated on CJ* but not exclusively so.

The transfer gap is task-structured, not model-structured. The locus of harm varies by task rather than by model family (Figure 2). Causal_judgement case studies reduce accuracy across all five models in the 3-seed mean: -4.2 pp (Claude), -3.5 pp (Llama), -2.3 pp (Gemini), -0.8 pp (GPT-4.1), and $+3.1$ pp (train model, mini), indicating near-universally harmful case studies. Geometric_shapes case studies produce a more heterogeneous pattern in the 3-seed mean (Δ_{cs}): mini

[1] RF scoring prompts the model to reason before giving a verdict. We observe a striking format sensitivity: switching from verdict-first to RF prompting raises web_of_lies accuracy from 42–66% to 99–100% across models. A dedicated appendix table will report this per-model and per-task.

[2] We use gpt-4.1 (not gpt-4.1-mini) for CS-ICL generation to give the baseline maximum quality, matching the approach in prior cheatsheet-ICL work Honda et al. [2025].

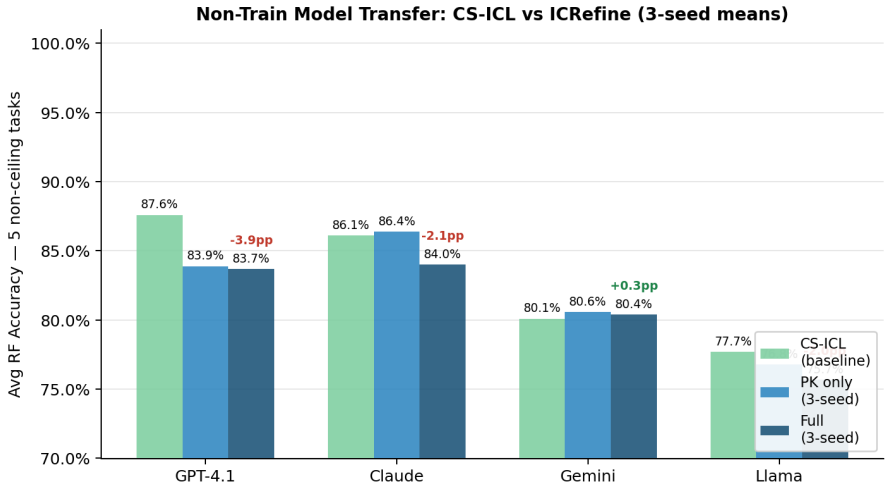

Figure 1: Non-train RF accuracy averaged over five non-ceiling tasks for each evaluation model under three conditions. PK-only transfers more consistently than the full cheatsheet across three of four model families, identifying Phase 2 case studies as the transfer bottleneck.

Table 2: Bidirectional transfer check: Gemini-2.0-flash as train model, full v3 pipeline, 3-seed means (seeds 1–3). Non-train 5-task RF averages compared to mini-trained v3 (3-seed mean) and CS-ICL. Δ_{cs} is full minus PK-only for CJ and GS. Gemini* denotes the train model.

Model	**Gemini-trained**	**Mini-trained (v3)**	**CS-ICL**	**Δ**
mini	80.9%	—	—	—
GPT-4.1	83.4%	83.7%	87.1%	−3.7
Claude-3.7	84.8%	84.0%	87.7%	−2.9
Llama-3.3	74.8%	75.7%	78.3%	−3.5
Gemini-2.0*	—	80.4%	81.4%	—

Δ = Gemini-trained full − CS-ICL (non-train only). 3-seed means.
CJ Δ_{cs}: −0.4 pp (GPT), +1.5 pp (Claude), −2.3 pp (Gemini*), −0.8 pp (Llama).
GS Δ_{cs}: +4.0 pp (GPT), +11.3 pp (Claude), +9.7 pp (Gemini*), −5.3 pp (Llama).

+1.3 pp, GPT-4.1 −0.7 pp, Claude −3.3 pp, Gemini +2.0 pp, Llama −6.7 pp. This pattern is consistent with models sharing the same SVG parsing failure modes (mini, Gemini) benefiting while more capable spatial reasoners (Claude) are hurt. The pattern is task-structured rather than a uniform model-size or model-family effect, suggesting that case study transferability depends on whether the failure mode the case study encodes is shared across model families. Formal, domain-structural failures (arc rotation classification, multi-subpath vertex counting) are more likely to be shared; oracle-contaminated reasoning failures (a paraphrase of one training item's gold chain-of-thought) mislead all models that encounter them.

A second training model yields consistent results. We replicated the gap with gemini-2.0-flash as train model (Table 2; 3-seed means). The non-train Δ vs. CS-ICL under Gemini training (−3.7, −2.9, −3.5 pp for GPT-4.1, Claude, Llama) are within 1 pp of the mini-trained equivalents (−3.4, −3.7, −2.6 pp), and all non-train models remain below CS-ICL under both training regimes. The task-structured pattern also replicates: CJ case studies are mostly negative regardless of train model, while GS case studies benefit models that share the train model's SVG parsing failures (Claude +11.3 pp under Gemini training vs. −3.3 pp under mini training). Two training models cannot establish task-determination as a universal law, but the replication across such different generators is striking.

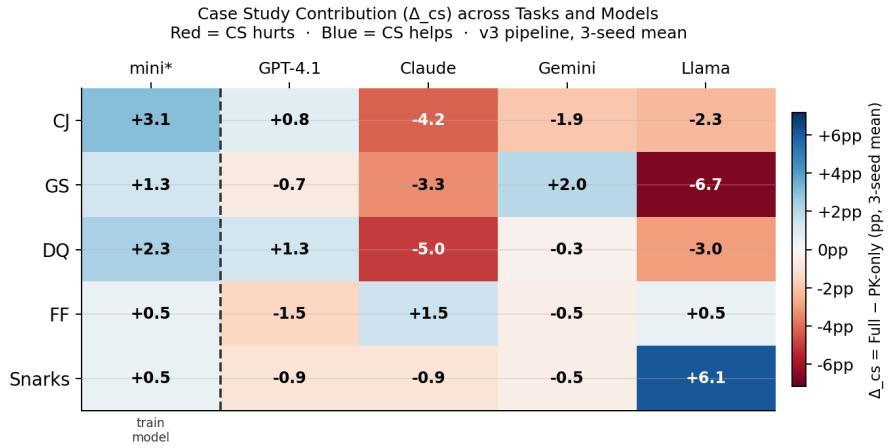

Figure 2: Δ_{cs} (full minus PK-only, in pp) heatmap over 5 tasks $\times$ 5 models, from v3 3-seed means. Blue cells: case studies help; red cells: case studies hurt. CJ shows uniformly negative case study contributions across all models (except mini, the train model), consistent with the failure modes characterised in Section 6.2. GS shows heterogeneous effects: positive for mini ($+1.3$ pp) and Gemini ($+2.0$ pp), negative for Claude (-3.3 pp) and Llama (-6.7 pp), consistent with model-specific SVG parsing capabilities. Dashed line separates the train model (mini) from non-train models. DQ (Claude -5.0 pp) and Llama GS (-6.7 pp) are the largest individual harms. FF is near-neutral across all models. Snarks is near-neutral except for Llama ($+6.1$ pp), whose PK-only accuracy on Snarks is below ceiling (83.1%), leaving room for case study benefit.

Table 3: Phase contribution on the test set—gpt-4.1-mini, reasoning-first (RF) accuracy. *Phase 0*: auto-bootstrap only. *Phase 1 (PK)*: after iterative PK patching. *Phase 2 (full)*: with case studies. Δ P1$\rightarrow$P2 isolates the marginal case study contribution. Single-run results.

Task	**CS-ICL**	**Phase 0**	**Phase 1**	**Phase 2**	**Δ P1$\rightarrow$P2**
Bool	100.0%	100.0%	100.0%	100.0%	0.0
CJ	66.7%	73.6%	70.1%	64.4%	-5.7
DU	95.0%	94.0%	91.0%	93.0%	$+2.0$
DQ	91.0%	84.0%	87.0%	86.0%	-1.0
FF	95.0%	96.0%	96.0%	97.0%	$+1.0$
GS	79.0%	70.0%	71.0%	76.0%	$\mathbf{+5.0}$
LD3	100.0%	100.0%	100.0%	100.0%	0.0
Nav	100.0%	100.0%	100.0%	100.0%	0.0
Snarks	95.8%	95.8%	94.4%	95.8%	$+1.4$
Sports	98.0%	100.0%	99.0%	100.0%	$+1.0$
WOL	100.0%	100.0%	100.0%	100.0%	0.0

5.2 On-train performance

Table 3 (Figure 6 in Appendix A) shows the phase-level breakdown on the train model. Seven of eleven tasks saturate at $\geq$95% after Phase 0 alone; iterative refinement only matters on four tasks. Phase 2 adds $+5.0$ pp for GS (geometric rules that transfer partially) and *subtracts* -5.7 pp for CJ (oracle-contaminated case studies that harm all models). The key lesson from on-train performance is cautionary. A $+5.0$ pp Phase 2 gain on the train model for GS corresponds to -3.3 pp (Claude) and -6.7 pp (Llama) at transfer time, and the -5.7 pp CJ Phase 2 penalty propagates identically to non-train models. Train-model accuracy gains during Phase 2 are a *poor predictor* of transfer quality.

6 Analysis

6.1 Mechanism 1: case study partition overfitting

Phase 2 partitions the training set by error type and generates one case study per coherent failure cluster. This mechanism succeeds when failures concentrate in reproducible, model-independent error classes. It fails, sometimes badly, when failures are dispersed or when the generated case studies encode the train model's idiosyncratic reasoning style rather than a transferable rule.

Geometric shapes: generalizable structural case studies. Listing 1 shows the two GS case studies produced by v3. Both ACTIVATE IF conditions encode structural geometric rules: multi-subpath vertex counting and arc rotation misclassification. Unlike the CJ case study (which paraphrases a specific oracle trace), these rules are intended to be structurally generalizable. The equal-radii-plus-nonzero-rotation condition for arc/circle distinction applies to any model encountering SVG arc paths, not just gpt-4.1-mini. One caveat: the `n_vertices` $\approx$ 7 predicate in CS1 is narrow; its generalizability depends on whether GS test failures cluster around similar vertex counts, which we have not directly verified. This structural character explains why Phase 2 adds $+5.0$ pp on the train model (Table 3). In the 5-model, 3-seed transfer evaluation, GS case studies are near-neutral on average: Gemini benefits most ($+2.0$ pp) while Claude (-3.3 pp) and Llama (-6.7 pp) are hurt. Models that already parse SVG paths capably are confused by case studies optimised for the train model's failure modes. The full pipeline falls below the CS-ICL baseline (74.0% vs. 79.0%, 3-seed means) because Phase 1 PK patching is limited on GS (Phase 1 achieves 72.7%, only $+2.7$ pp above Phase 0), and the Phase 2 gain for stronger models cannot offset the harm to weaker ones.

```
=== CS1: Correct vertex counting across multiple "M" subpaths ===
ACTIVATE IF:
  has_multi_subpath = true
  n_vertices ~= 7
  error = miscounted_vertices
WHY: Multiple M commands may form one connected polygon; endpoints shared
     across subpaths must be counted collectively, not per-subpath.

=== CS2: Arc paths misclassified as circles instead of ellipses ===
ACTIVATE IF:
  has_arc = true
  arc_radii_equal = true
  rotation_angle significantly non-zero
  model_predicts = circle
WHY: Equal radii alone do not classify an arc as a circle; the rotation
     angle must be near-zero for circle classification.
```

Listing 1: Phase-2 case studies generated for geometric_shapes (v3). ACTIVATE IF conditions encode structural SVG parsing rules intended to generalise across model families (narrow predicates such as `n_vertices`$\approx$7 limit scope): under Gemini training, these rules benefit GPT-4.1 ($+4.0$ pp) and Gemini ($+5.0$ pp).

Formal fallacies: partition search fixation. Four case studies were generated for formal_fallacies, but three (CS1, CS2, CS4) all address illicit conversion of conditionals (premise: $A \Rightarrow B$; conclusion: $B \Rightarrow A$); CS1 and CS4 are near-identical. This redundancy reveals a limitation in the greedy partition search: Phase 2 encounters the illicit-conversion failure type across multiple bins and generates a new case study each time without detecting prior overlap. Only CS3 (invalid inference from a disjunction premise) covers orthogonal ground. Deduplication, by checking new case study ACTIVATE IF conditions against existing ones before accepting, would resolve this. It is a straightforward pipeline fix we leave for future work.

Causal judgement: oracle-contaminated specificity. Causal_judgement has the most dispersed failure distribution: 35 training failures spanning joint-AND causation, OR/redundant causes, prevention chains, norm violations, timing effects, and intentionality judgements. No single partition is large enough to yield a high fix-rate case study without regressing others. The single CJ case study that passed the acceptance gate (fix_rate = 42%) is shown in Listing 2: a free-form narrative that closely paraphrases one specific training item's oracle reasoning. *Note: this seed 1 case study*

is atypical. Seeds 2 and 3 generate structurally formatted ACTIVATE-IF case studies rather than free-form narratives, yet both also reduce accuracy (Appendix B), implicating failure-distribution heterogeneity as a co-cause independent of oracle format. This oracle-derived specificity is a plausible primary contributor to the -5.7 pp Phase 2 penalty: the case study encodes reasoning steps for one idiosyncratic joint-AND scenario, misleading the model on unrelated causal question types. An alternative co-explanation is CJ's inherently heterogeneous failure distribution: 35 training failures spanning joint-AND causation, OR/redundant causes, prevention chains, norm violations, timing effects, and intentionality judgements. This diversity makes any single case study prone to overfitting regardless of oracle access. The oracle ablation in Section 6.2 tests whether removing oracle access reduces the Phase 2 penalty; the residual gap after oracle removal provides evidence on which factor dominates. The cross-seed case study inventory (Appendix B) further characterises this: seeds 2 and 3 produce case studies that are *not* oracle-contaminated in form, yet still reduce accuracy, implicating failure-distribution heterogeneity as a co-cause. Critically, this harm is not limited to the train model. The 5-model 3-seed evaluation shows Phase 2 reducing accuracy for Claude (-4.2 pp), Gemini (-1.9 pp), and Llama (-2.3 pp); GPT-4.1 shows a marginal $+0.8$ pp gain (Section 6.2 and Figure 2).

```
=== Joint causes (logical AND): garden fertilizers ===
Scenario: Two gardeners independently apply fertilizers; plants dry out
  only if BOTH fertilizers are applied simultaneously.
Verdict: NO (neither gardener alone caused it)
Apply when: outcome requires simultaneous co-occurrence of conditions
  (logical AND)---neither party alone is causally responsible.
```

Listing 2: The single Phase-2 case study generated for causal_judgement (v3). Free-form prose paraphrasing one training item's oracle reasoning, encoding the train model's idiosyncratic treatment of joint-AND causation.

6.2 Mechanism 2: oracle contamination

Phase 2 injects the gold chain-of-thought (`item["reason"]`) into failure-analysis prompts during case study generation. Experiment E3 tests the causal role of oracle access by re-running the CJ pipeline with both Phase 1 and Phase 2 oracle contexts disabled (`-no-oracle`).

Oracle-free Phase 2 generates more transferable case studies. Unlike an earlier CoT-mode ablation that disabled only the Phase 2 oracle and produced zero case studies, the RF-mode E3 condition (`-no-oracle` on both phases) still generates Phase 2 case studies. The model synthesises correction rules from prediction failures alone, without gold reasoning. The result was striking: every single model improves under E3 (Table 4 and Figure 3), using 3-seed means for both conditions. Improvements range from $+1.9$ pp (mini, Gemini) to $+3.8$ pp (Llama), with GPT-4.1 gaining $+3.1$ pp and Claude $+3.5$ pp. This consistent direction across *all five models* and *all three pipeline seeds* is the clearest causal signal in our evaluation: oracle injection is a genuine *contributing factor* in the CJ case study's harm. Effect sizes are moderate (≤ 4 pp); the benefit of removing oracle access is real but not sufficient to close the gap. E3 CJ does *not* fully recover CS-ICL performance: the non-train E3 average (68.5%) remains 3.9 pp below the CS-ICL CJ average (72.4%), comparable in size to the oracle-removal effect itself ($+1.9$–$+3.8$ pp). This residual gap is consistent with heterogeneous failure distribution being a co-contributing cause: even oracle-free case studies may overfit when CJ failures span six or more structurally distinct subtypes. Oracle contamination removal narrows but does not close the gap; CJ case study generation remains difficult regardless of oracle access.

The oracle dilemma has a design implication. Oracle injection may assist Phase 2 case study generation on tasks with heterogeneous failure distributions, but simultaneously risks encoding reasoning specificity that does not transfer. The E3 result suggests that for CJ specifically, oracle-free generation already produces adequate case study material. The gold chain-of-thought is not required to identify generalizable correction rules, and providing it causes overfit to specific oracle reasoning styles. The contrast with GS (where oracle-assisted case studies encode formally generalizable geometric rules) indicates that the appropriate use of oracle context depends on whether the failure mode admits a principled, item-independent correction rule. Notably, seeds 2 and 3 of the v3 pipeline generate structurally general, non-oracle-contaminated case studies for CJ (Appendix B), yet these seeds still reduce transfer accuracy below CS-ICL. This confirms that failure-distribution heterogeneity is an independent contributing factor beyond oracle contamination. The CJ failure

Table 4: E3 oracle ablation on causal_judgement (RF, 5 models). E3: both Phase 1 and Phase 2 oracle disabled (`-no-oracle`); Phase 2 generates case studies without gold chain-of-thought. v3 full: standard oracle-enabled pipeline. Δ = E3 full − v3 full. **Both columns are 3-seed means** (E3 and v3 each averaged over 3 independent pipeline runs); improvements range from $+1.9$ to $+3.8$ pp. The train model (mini*, $+1.9$ pp) and Gemini ($+1.9$ pp) improvements approach the 2–4 pp noise floor; non-train improvements for GPT-4.1 ($+3.1$ pp), Claude ($+3.5$ pp), and Llama ($+3.8$ pp) are more robustly above it. * denotes the training model.

Model	E3 (no oracle)	v3 full	Δ
gpt-4.1-mini*	69.7%	67.8%	+1.9
gpt-4.1	70.5%	67.4%	+3.1
claude-3.7	69.0%	65.5%	+3.5
gemini-2.0	65.9%	64.0%	+1.9
llama-3.3	68.6%	64.8%	+3.8

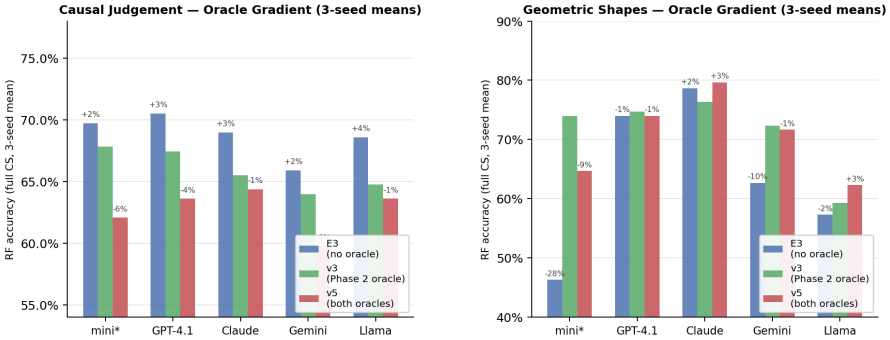

Figure 3: Oracle contamination gradient on causal_judgement (left, full CS accuracy) and geometric_shapes (right, full CS accuracy) across three oracle conditions: E3 no-oracle, v3 standard (Phase 2 oracle only), and v5 full-oracle (both Phase 1 and Phase 2 oracle enabled). All values are 3-seed means. On CJ, removing oracle access consistently improves transfer: E3 > v3 for 4 of 5 models; full oracle (v5) degrades accuracy further for GPT-4.1, Gemini, and Llama. On GS, oracle effects are less consistent, reflecting that GS case study harm is structural (SVG parsing) rather than oracle-contamination driven.

space (35 instances spanning ≥ 6 structurally distinct causal sub-types) is simply too diverse for case study-based correction to generalise reliably, regardless of oracle access. Figure 4 confirms this: the within-condition seed-to-seed variance on GS is high and the oracle ordering is non-monotone, whereas on CJ the E3 improvement is consistent across all seeds (Figure 9).

6.3 Mechanism 3: bootstrap convergence and sequential patching limits

Phase 1 applies sequential PK patching: each iteration generates a single candidate revision, accepts or rejects it on accuracy delta against the current PK, and moves on. When the bootstrap PK already satisfies the acceptance gate (because the LLM generates a sufficiently accurate initial cheatsheet from 75 training examples), Phase 1 applies zero patches. We observe this on five ceiling tasks (Bool, LD3, Nav, WOL, FF) where Phase 0 and Phase 1 accuracy are equal to within evaluation noise ($\geq$96% throughout), and on geometric_shapes where Phase 1 adds only $+1.0$ pp above Phase 0. On CJ and DQ, Phase 1 does apply revisions but the gains are small: DQ improves $+3.0$ pp from Phase 0 to Phase 1 (both single-run). CJ shows the opposite effect: the 3-seed Phase 1 mean (64.8%) falls 8.8 pp below the single-run Phase 0 bootstrap (73.6%). To avoid conflating single-run and 3-seed estimates, we note that all three individual Phase 1 seeds also fall below this single-run baseline (63.2%, 66.7%, 64.4%), confirming a genuine Phase 1 failure rather than a comparison artefact. This is consistent with the oracle contamination finding in Section 6.2: Phase 1 failure signals on CJ are

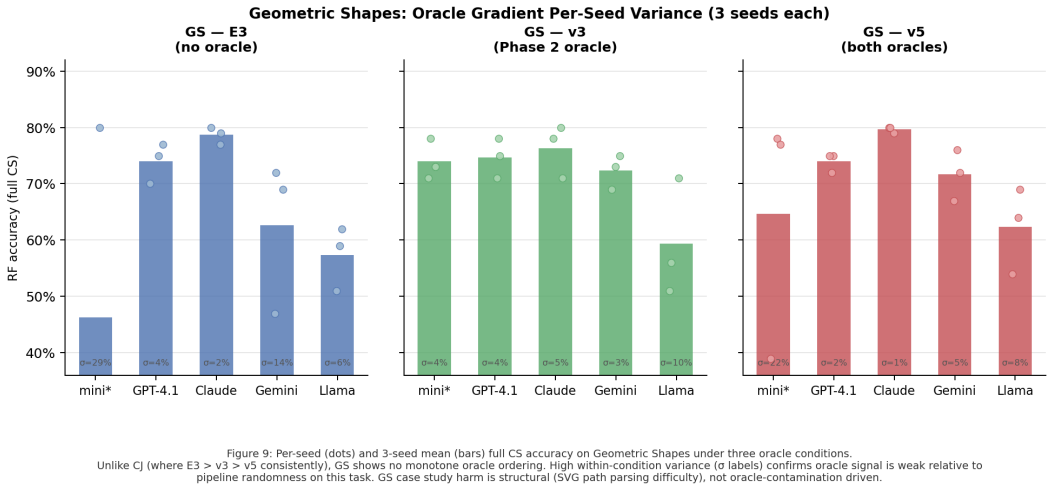

Figure 4: Per-seed (dots) and 3-seed mean (bars) full CS accuracy on geometric_shapes under E3 (no oracle), v3 (Phase 2 oracle), and v5 (both oracles). Unlike causal_judgement (where E3 $>$ v3 holds for all seeds and all models), GS shows no consistent oracle ordering. Within-condition σ values (bottom labels) are 3–10 pp, comparable to the cross-condition differences, confirming that oracle injection provides no reliable signal on this task. GS case study harm is structural (SVG path parsing difficulty) rather than oracle-contamination driven.

oracle-contaminated, so iterative patching reinforces idiosyncratic reasoning patterns rather than improving generalizable rules.

Table 3 (Figure 6 in Appendix A) confirms that Phase 2 is the variance-introducing step: it produces the largest positive contribution ($+5.0$ pp on GS) and the largest negative contribution (-5.7 pp on CJ).

6.4 Intervention: evolutionary algorithm Phase 1

To address bootstrap convergence, we introduce an **Evolutionary Algorithm (EA) Phase 1** that replaces single-candidate patching with population-based search: at each generation, three candidate PK rewrites are scored in parallel on a held-out validation set (20% of training), and the top two survivors are recombined into the next generation via crossover.

Table 5 compares EA vs. standard Phase 1 on RF test accuracy using the PK-only cheatsheet (prior knowledge without case studies), which isolates Phase 1 quality from Phase 2 effects.

EA addresses the GS bootstrap convergence case. Standard Phase 1 applies zero patches to GS (the bootstrap PK already satisfies the acceptance gate), yielding a PK-only test accuracy of 72.7% averaged over three seeds, a full 6.3 pp below the CS-ICL reference. EA Phase 1 breaks through: three patches are applied across generations, raising PK-only test accuracy to 78.7% ($+6.0$ pp, 3-seed mean) and essentially matching the CS-ICL baseline (79.0%) on the train model. What surprised us most was the downstream consequence: EA's improved prior knowledge is comprehensive enough that the Phase 2 acceptance gate rejects *all* case study candidates: zero case studies are generated, vs. two in the standard pipeline. The EA pipeline is purely PK-based for GS. Evolutionary search encoded vertex-counting and arc-classification corrections directly in prior knowledge at sufficient quality that no case study is needed. The EA advantage is consistent across all three seeds and all five evaluation models (Figure 9), and EA also substantially reduces seed-to-seed variance (Llama $\sigma = 4.7\%$ vs. 10.4%), indicating a structurally cleaner cheatsheet.

Results on other tasks. EA also raises CJ PK accuracy by $+5.4$ pp (3-seed mean), but the EA full-pipeline on CJ is *lower* than standard (64.4% vs. 67.8%): the four EA-generated CJ case studies are oracle-derived in content and hurt transfer, reinforcing Section 6.2. The EA+no-oracle combined pipeline (Section 6.5) applies EA Phase 1 PK alongside oracle-free Phase 2 case studies for CJ,

Table 5: EA Phase 1 vs. standard Phase 1—RF test accuracy on gpt-4.1-mini. *PK only*: cheatsheet without case studies evaluated on test set. *Full*: complete pipeline (PK + Phase 2 case studies). EA: population size 3, 2 survivors, 12 K-char PK budget, 20% hold-out. Std CS / EA CS: number of Phase 2 case studies retained in each condition. **All PK columns are 3-seed means** (seeds 1–3 for both conditions); CS counts and patch counts are from seed 1.

Task	Std PK	Std CS	EA PK	EA CS	EA patches	Δ PK
CJ	64.8%	1	**70.1%**	4	0	+5.4
GS	72.7%	2	**78.7%**	0	3	+6.0
Snarks	96.7%	2	95.8%	1	0	−0.9
DQ	84.7%	0	84.7%	3	1	0.0

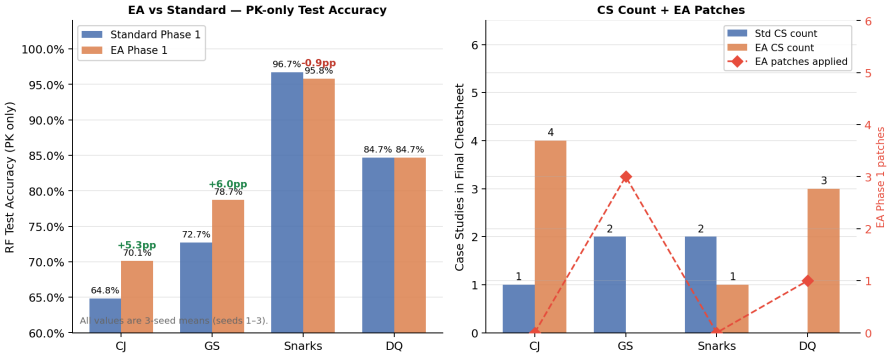

Figure 5: EA Phase 1 vs. standard Phase 1—RF test PK-only accuracy for four non-ceiling tasks (all 3-seed means, train model). GS is the primary beneficiary (+6.0 pp), eliminating Phase 2 case studies entirely. CJ shows a consistent +5.4 pp PK improvement; Snarks and DQ are within noise (−0.9 pp and 0.0 pp respectively).

addressing both failure modes simultaneously. On Snarks and DQ, EA is within noise (−0.9 pp and 0.0 pp); we do not claim EA as a general improvement beyond the GS bootstrap convergence case.

6.5 Combined interventions: does the best-available pipeline beat CS-ICL?

We address the central diagnostic question with two comparisons. First, **EA+no-oracle**: a single coherent pipeline applying EA Phase 1 and oracle-free Phase 2 across all five tasks uniformly (3-seed means). Second, **best-combo**: a per-task oracle that selects the optimal intervention independently per task (EA PK-only for GS; oracle-free E3 for CJ; v3 full for FF, DQ, Snarks). Best-combo is not deployable: it requires knowing which intervention wins per task without evaluation labels. It is best understood as an informal upper bound rather than a rigorous one, as selecting the best of 3 candidate interventions across 5 tasks introduces multiple-comparisons inflation that is non-trivial relative to the 2–4 pp noise floor.

Table 6 reports both. EA+no-oracle remains below CS-ICL for all four model families: GPT-4.1 (−3.3 pp), Claude (−2.4 pp), Gemini (−0.6 pp), and Llama (−3.6 pp). Bootstrap CIs (95%, task-resampling) exclude zero for GPT-4.1 and Claude; Gemini and Llama CIs cross zero. Best-combo adds 0.1–2.9 pp over EA+no-oracle depending on the model, confirming that the per-task oracle margin is small. All four families improve over the uncorrected v3 full pipeline under both comparisons.

Three cautions apply. First, the CJ intervention (E3) improves *over v3* for all models but does not reach CS-ICL on CJ specifically: E3 CJ non-train average is 68.5% vs. CS-ICL CJ 72.4%; oracle contamination removal narrows the gap but does not close it. Second, EA+no-oracle does not reach CS-ICL parity for GPT-4.1 (−3.3 pp) or Llama (−3.6 pp); for these model families the residual transfer gap, driven predominantly by CJ and a Llama-specific GS regression, is not addressable by the interventions studied here. Third, all averages are backed by 3-seed means throughout; measurement noise is not the primary uncertainty.

Table 6: EA+no-oracle and best-combo vs. CS-ICL—non-train 5-task RF average (3-seed means). EA+no-oracle is a single coherent pipeline (EA Phase 1, oracle-free Phase 2) applied uniformly across all five tasks. Best-combo applies the best available intervention per task (EA PK-only for GS; E3 full for CJ; v3 full for FF/DQ/Snarks). Δ = EA+no-oracle − CS-ICL; 95% CI bootstraps over 5 tasks. * CI excludes zero. † Best-combo is a per-task oracle, not a deployable pipeline. **CS-ICL column is the same 3-seed mean as Table 1.**

Model	CS-ICL	EA+no-oracle	Δ	Best-combo†	95% CI
GPT-4.1	87.1%	83.8%	-3.3	84.9%	$[-6.3, -1.6]^*$
Claude-3.7	87.7%	85.3%	-2.4	85.4%	$[-5.4, -0.0]^*$
Gemini-2.0	81.4%	80.8%	-0.6	81.2%	$[-2.7, +1.6]$
Llama-3.3	78.3%	74.7%	-3.6	77.6%	$[-6.4, +0.8]$

6.6 Seed variance analysis

API evaluations show 2–4 pp across-run non-determinism at temperature zero. We ran v3, E3, and EA three times each from independent API calls and report 3-seed means throughout; per-seed results are shown in Figure 9 (Appendix C). Both key findings are robust: the EA GS improvement holds in all three seeds for 4 of 5 models, and EA reduces Llama seed variance from $\sigma = 10.4\%$ to $\sigma = 4.7\%$, indicating a structurally cleaner cheatsheet. The E3 oracle-free improvement on CJ is positive for every model in every seed, making it the strongest cross-seed result in the evaluation. Single-run E3 numbers for GPT-4.1 and Llama ($+8.1$ pp and $+8.0$ pp) were inflated because the corresponding v3 seeds were unusually weak; the 3-seed means ($+3.1$ pp and $+3.8$ pp) are the reliable estimates.

7 Discussion

Our results point toward several generalizable principles for iterative cheatsheet design that extend beyond the specific pipeline and task suite studied here.

Principle 1: Task failure structure, not model identity, governs transferability. The transfer deficit of ICREFINE over CS-ICL is nearly identical whether the train model is gpt-4.1-mini or gemini-2.0-flash (-3.4 vs. -3.7 pp for GPT-4.1; -3.7 vs. -2.9 pp for Claude). The locus of harm is also consistent across training directions: CJ case studies are universally harmful, GS case studies benefit models that share the train model's SVG parsing failures. This suggests that the key design variable is *failure mode coverage*: whether the failure modes encoded in a case study are broadly shared across model families. This matters more than the identity of the model used to generate the cheatsheet. Case study transferability is therefore a *property of the task* (specifically, whether its failures are structurally universal or model-specific) rather than a property of the training regime. Practitioners should audit whether target-model failure modes overlap with train-model failure modes before deploying iteratively refined cheatsheets.

Observation 2: On the two tasks with non-trivial Phase 2 effects, oracle context helps where failure modes are principled and hurts where they are idiosyncratic. Oracle injection (gold chain-of-thought alongside each training failure) was intended to help the train model identify better correction rules. It produces opposite effects on our two most informative tasks: GS case studies generated with oracle access encode structurally transferable geometric rules (vertex counting, arc classification), while CJ case studies encode idiosyncratic reasoning traces for one specific causal scenario. The oracle is not itself to blame. The CJ E3 intervention shows that oracle-free generation narrows but does not close the gap, because heterogeneous failure distribution is a co-cause. This observation rests on exactly two task-level data points (GS and CJ); whether the principled-vs-idiosyncratic distinction generalises as a design rule is an empirical question for future work. Provisionally, it suggests that oracle context is worth including where failures admit a compact, item-independent correction, and worth avoiding where the failure space is structurally diverse.

Principle 3: A strong single-pass generator may extract holistic heuristics that iterative patching cannot recover. CS-ICL, generated by gpt-4.1 in a single pass over all 100 training examples,

consistently outperforms ICREFINE (train model: gpt-4.1-mini) on the four non-train evaluation models. Part of this gap reflects generator quality (gpt-4.1 vs. gpt-4.1-mini), and a mini-generated CS-ICL ablation would isolate this confound. But we believe a more fundamental hypothesis also operates: a model reading all 100 training examples in a single, unconstrained pass may extract broadly applicable task intuitions that no sequence of local failure-focused patches recovers. We call this the *holistic-vs-local* hypothesis: iterative refinement is locally optimal but globally suboptimal when failure modes are heterogeneous and the full training set jointly encodes cross-cutting heuristics invisible from any single failure slice. Testing this directly, e.g., by ablating the bootstrap PK input size or comparing failure-focused vs. full-set single-pass generation, is a concrete direction for future work. Regardless of which mechanism dominates, a strong single-pass generator should be the first baseline before investing in iterative refinement.

Principle 4: Partition overfitting is a structural failure of greedy partition search. Phase 2's greedy partition search re-encounters the same failure type across multiple bins and generates redundant case studies without detecting the overlap (formal fallacies: three of four case studies address illicit conditional conversion; CJ seeds 2–3 converge on joint-AND causation from three different angles). This is not a failure of the generation model; it is a failure of the search procedure. Deduplication via ACTIVATE IF similarity checks before acceptance, and diversity-promoting objectives in case study selection, would mechanically address this issue. The parallel with best-of-N selection failures in the size ablation (test ordering reverses from train on GS) suggests a broader lesson: train-time acceptance metrics (fix-rate, regression-rate) are necessary but not sufficient conditions for test-time generalization of case studies.

Principle 5: Evolutionary search adds value when greedy search stalls, at least on the task where we observe stalling. EA Phase 1 improves PK accuracy on the two tasks where sequential patching stalls (GS: $+6.0$ pp; CJ: $+5.4$ pp) and is within noise on tasks that sequential patching already handles (Snarks, DQ). On GS specifically, EA's improved prior knowledge is comprehensive enough that Phase 2 case studies are rejected entirely. This is a qualitative change in pipeline behaviour, not just an accuracy increment. We do not claim EA as a general replacement for sequential patching: our evidence comes from a single bootstrap-stalled task (GS), and whether population-based search reliably helps on other stalled tasks is an open question. Subject to that caveat, the evidence motivates a cascade design: use single-candidate patching until stall, then escalate to population-based search. The additional cost of evolutionary search is then incurred only where it is causally necessary.

8 Related Work

In-context learning. In-context learning (ICL) emerged as a central capability of large language models with GPT-3 [Brown et al., 2020], which showed that a few labeled examples prepended to a prompt suffice for strong zero-shot-equivalent task performance. Subsequent work established that demonstration selection, ordering, and format can each produce large accuracy swings independent of model capability [Zhao et al., 2021, Lu et al., 2022]. Chain-of-thought (CoT) prompting [Wei et al., 2022] extended ICL to multi-step reasoning by eliciting intermediate reasoning traces, and many-shot ICL [Agarwal et al., 2024] showed that scaling the number of demonstrations continues to improve performance up to the context window. ICREFINE operates in the complementary regime: rather than maximizing the number of demonstrations, it compresses task knowledge into a fixed-length cheatsheet that is prepended to all test inputs, reducing per-input token cost while retaining task-specific guidance.

Cheatsheet and prompt distillation. Honda et al. [2025] introduced Cheatsheet-ICL (CS-ICL), the direct baseline for our work: they show that a single-pass natural-language ruleset generated from training demonstrations matches or exceeds many-shot ICL accuracy at a fraction of the inference cost. Related work on prompt distillation [Li et al., 2023] and task-specific instruction generation [Zhou et al., 2023, 2022] similarly attempts to compress demonstration knowledge into reusable prompts. ICREFINE extends CS-ICL with iterative refinement and a structured case study component, and contributes a cross-model transfer evaluation that CS-ICL does not perform.

Automatic prompt optimization. Treating prompt construction as an optimization problem has a rich recent history. APE [Zhou et al., 2023] and OPRO [Yang et al., 2024] generate and rank

candidate prompts using LLM proposals; ProTeGi [Pryzant et al., 2023] introduces natural-language "gradient" critiques of failures as directed edits. More recent systems raise the ambition further. DSPy [Khattab et al., 2024] compiles declarative LLM program specifications into self-improving pipelines that jointly optimize prompts and few-shot examples end-to-end. TextGrad [Yuksekgonul et al., 2024] treats text outputs as "differentiable" objects, propagating failure feedback as natural-language gradients through multi-step LLM pipelines. PromptBreeder [Fernando et al., 2024] applies evolutionary search directly to task prompts and prompt-mutation operators, producing self-referential improvement loops. ICREFINE's Phase 1 is most closely related to ProTeGi-style failure-directed editing, but operates on a structured two-component cheatsheet (PK + case studies) rather than a free-form instruction. Our EA Phase 1 variant is related to PromptBreeder's population-based mutation, but focuses on the specific bottleneck of bootstrap convergence and uses a validation-gate stopping criterion motivated by the regression-regression trade-off in cheatsheet revision. Unlike DSPy and TextGrad, which optimize prompts for a fixed evaluation model, our primary question concerns cross-model *transfer*: whether a cheatsheet optimised against one model family generalises to another.

Iterative and self-improving NLP systems. Iterative self-refinement [Madaan et al., 2024] uses the model's own feedback to revise outputs across multiple rounds; Reflexion [Shinn et al., 2024] accumulates verbal experience from failures in episodic memory. Both refine individual *outputs* at inference time; ICREFINE instead refines a *shared cheatsheet* at train time, applied read-only at inference time. STaR [Zelikman et al., 2022] iteratively bootstraps reasoning chains from correct self-generated rationales; the sample-and-accept gating in Phase 1 is conceptually related but applied to cheatsheet revision rather than rationale generation. Constitutional AI [Bai et al., 2022] and InstructGPT [Ouyang et al., 2022] use feedback to improve model behavior, but via weight updates rather than prompt edits.

Cross-model prompt transfer. Soft-prompt tuning [Lester et al., 2021] encodes task knowledge in continuous token embeddings, which are model-specific by construction. Hard (discrete) prompt optimization [Zhou et al., 2023, Yang et al., 2024] is more portable in principle, but transfer remains underexplored, as most prompt optimization papers evaluate on the same model used for optimization [Chen et al., 2023]. Logan *et al.* [Logan IV et al., 2021] study cross-model generalization of few-shot prompts, finding that simpler prompts transfer more reliably. Our contribution to this thread is a controlled cross-model transfer evaluation of *iteratively refined* cheatsheets, showing that the cheatsheet's quality on the train model is a poor predictor of transfer quality. The task's failure mode structure, not the model family, is the primary determinant of transferability.

9 Conclusion

We introduced ICREFINE, a three-phase iterative pipeline for cheatsheet construction, and used it as a diagnostic instrument to characterize why iterative cheatsheet refinement fails to transfer across model families on BIG-Bench Hard. The full pipeline underperforms the static CS-ICL baseline on all four non-train evaluation families in point estimate, with statistically significant deficits for GPT-4.1 and Claude. Three mechanistically distinct failure modes (oracle contamination, partition overfitting, and bootstrap convergence limits) each map to specific tasks and persist regardless of which model generates the cheatsheet, confirming that the transfer gap is task-determined rather than model-determined.

Two targeted interventions, evolutionary Phase 1 and oracle-free Phase 2, partially address two of the three failure modes. EA Phase 1 closes the bootstrap convergence gap on geometric_shapes ($+6.0$ pp PK accuracy) and eliminates the need for Phase 2 case studies on that task entirely. Oracle-free Phase 2 reduces but does not eliminate the CJ transfer gap ($+1.9$–$+3.8$ pp), with the residual attributable to heterogeneous failure distribution that is orthogonal to oracle access. The combined EA+no-oracle pipeline remains below CS-ICL for GPT-4.1 and Llama, suggesting that the bottleneck is the difficulty of case study generalization under diverse failure distributions, not any single addressable design flaw.

These results, within the scope of five short-answer classification tasks from BIG-Bench Hard with 100 training examples each, suggest that the difficulty of iterative cheatsheet refinement is determined by task failure structure. Tasks with concentrated, structurally universal failure modes (such as the

SVG-parsing errors in geometric_shapes) appear amenable to iterative refinement and case study generation. Tasks with dispersed, model-specific, or idiosyncratic failure distributions (such as causal_judgement with its 35 failures spanning six causal sub-types) are not well-served by the approach studied here. Whether this pattern extends to longer-context tasks, open-ended generation, or larger training sets remains an open empirical question. The pipeline may perform differently in settings where failure modes are more enumerable or where the training set is large enough to give each partition sufficient support for reliable case study generalization. For practitioners in similar small-scale classification settings, a strong single-pass generator applied to all training examples should serve as the primary baseline before investing in iterative refinement. Train-model accuracy gains during refinement do not reliably predict cross-model transfer quality. We release our pipeline code, cheatsheets, and evaluation logs to support follow-on work.

10 Limitations

The bidirectional transfer check (Section 5.1, Table 2) is based on two training models; the task-determined framing would be strengthened by a third. BIG-Bench Hard test sets contain 87–250 examples per task, meaning a 1 pp difference corresponds to 1–2 examples; differences below 3 pp should be interpreted cautiously.

CS-ICL baseline. CS-ICL is now re-scored over three independent seeds with the same RF prompting as all ICREFINE conditions, resolving the earlier single-run asymmetry. Differences between the 3-seed CS-ICL means and the original single-run values are ≤ 1.3 pp, confirming that CS-ICL evaluation is stable across seeds. CS-ICL is generated by gpt-4.1 (not gpt-4.1-mini), so part of the residual gap may reflect generator quality rather than the static-vs-iterative design distinction; a mini-generated CS-ICL ablation would isolate these factors.

Scale and significance. Several key claims rest on single-task diagnostic observations (oracle contamination on CJ, bootstrap convergence on GS); confirming these on additional tasks would strengthen the mechanistic account. Task-level bootstrap CIs are now reported in Table 1 and Table 6; per-example bootstrap CIs are not computed and would further sharpen the headline claim before a venue submission. A larger-dataset replication of the full pipeline (magma, 954 training items) is in progress and may be reported in a separate companion submission.

References

Rishabh Agarwal, Aaditya Maharana, Jongwon Park, Xinyi Xue, Mehmet Abbas, Xingyi Yu, Karthik Narasimhan, Kyunghyun Cho, and Shivam Garg. Many-shot in-context learning. *arXiv preprint arXiv:2404.11018*, 2024.

Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, Nicholas Johnston, Saurav Kadavath, Jackson Kernion, Tom Conerly, Shady El-Showk, Nelson Elhage, Zac Hatfield-Dodds, Danny Hernandez, Tristan Hume, Josh Jacobson, Scott Lasenby, Liane Lovitt, Neel Nanda, Catherine Olsson, Amanda Amodei, Dario Amodei, Jack Clark, Sam McCandlish, Chris Olah, Jacob Steinhardt, Jascha Sohl-Dickstein, and Jared Kaplan. Constitutional AI: Harmlessness from AI feedback. 2022.

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901, 2020.

Qinyuan Chen, Zhengyan Hu, Xu Zhang, Yutai Hou, Zhiyuan Zhang, Yankai Ye, Guo-Bo Wang, Wen Zhang, Xu Han, Zhiyuan Liu, and Maosong Sun. Are prompts all the story? no. a comprehensive and broader view of instruction learning. *arXiv preprint arXiv:2303.10475*, 2023.

Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. Promptbreeder: Self-referential self-improvement via prompt evolution. In *Advances in Neural Information Processing Systems*, volume 37, 2024.

Ukyo Honda, Soichiro Murakami, and Peinan Zhang. Distilling many-shot in-context learning into a cheat sheet. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025. URL `https://arxiv.org/abs/2509.20820`.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Hira, Ashutosh Zhong, Tom Kwiatkowski, Christopher Potts, Percy Liang, and Matei Zaharia. DSPy: Compiling declarative language model calls into self-improving pipelines. In *The Twelfth International Conference on Learning Representations*, 2024.

Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 3045–3059, 2021.

Lei Li, Yongfeng Zhang, and Li Chen. Prompt distillation for efficient LLM-based recommendation. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*, pages 1348–1357, 2023.

Robert L. Logan IV, Ananth Balashankar, Minqing Chen, Sebastian Ruder, and Sameer Singh. Cutting down on prompts and parameters: Simple few-shot learning with language models. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pages 3044–3052, 2021.

Yao Lu, Max Bartolo, Alastair Moore, Sebastian Riedel, and Pontus Stenetorp. Fantastically ordered prompts and where to find them: Overcoming few-shot prompt order sensitivity. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8086–8098, 2022.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skye Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Sean Welleck, Peter Clark, Antoine Bosselut, Peter Clark, Sameer Singh, and Ari Holtzman. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, volume 36, 2024.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744, 2022.

Reid Pryzant, Dan Iter, Jerry Li, Yin Tat Lee, Chenguang Zhu, and Michael Zeng. Automatic prompt optimization with "gradient descent" and beam search. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 7957–7968, 2023.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, 2024.

Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc V. Le, Ed H. Chi, Denny Zhou, and Jason Wei. Challenging BIG-bench tasks and whether chain-of-thought can solve them. *arXiv preprint arXiv:2210.09261*, 2022.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837, 2022.

Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *The Twelfth International Conference on Learning Representations*, 2024.

Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. TextGrad: Automatic "differentiation" via text. *arXiv preprint arXiv:2406.07496*, 2024.

Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. STaR: Bootstrapping reasoning with reasoning. In *Advances in Neural Information Processing Systems*, volume 35, pages 15476–15488, 2022.

Zihao Zhao, Eric Wallace, Shi Feng, Dan Klein, and Sameer Singh. Calibrate before use: Improving few-shot performance of language models. In *Proceedings of the 38th International Conference on Machine Learning*, pages 12697–12706. PMLR, 2021.

Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. Large language models are human-level prompt engineers. arXiv preprint arXiv:2211.01910, 2022.

Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. Large language models are human-level prompt engineers. In *The Eleventh International Conference on Learning Representations*, 2023.

A Supplementary figures and ablations

A.1 Phase contribution breakdown (all 11 tasks)

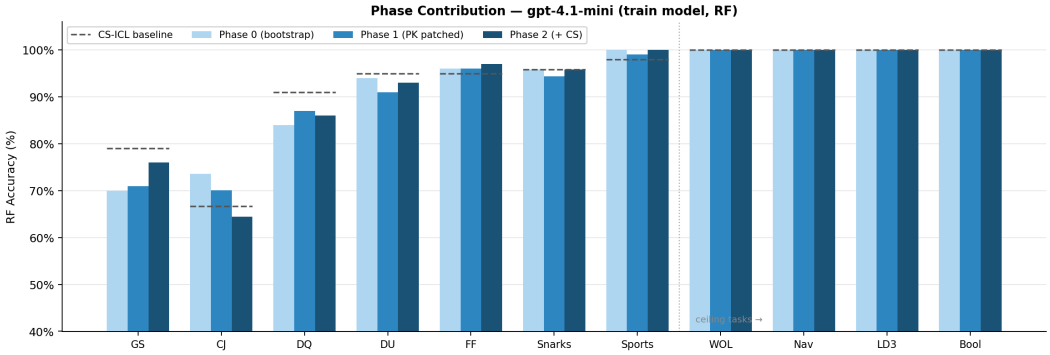

Figure 6: RF test accuracy by pipeline phase for gpt-4.1-mini across all 11 BIG-Bench Hard tasks. Seven tasks reach $\geq$95% after Phase 0 alone. Phase 2 produces the largest task-level shifts on geometric_shapes ($+5.0$ pp) and causal_judgement (-5.7 pp).

A.2 Cheatsheet size ablations (single-run, exploratory)

These ablations are exploratory single-run results on gpt-4.1-mini only. All differences fall within or near the 2–4 pp evaluation noise floor; we report them for completeness but draw no strong conclusions.

A.3 Phase 1 PK character limit

Table 7 and Figure 7 report RF test accuracy across four character-limit conditions (3 K, 6 K, 12 K, unlimited). The ordering is non-monotone across all four tasks: no single cap consistently outperforms the others. The observation that 3 K can match or exceed larger caps motivates the Phase 0 bootstrap size cap in ablation_size2.

A.4 Phase 2 best-of-N case study selection

The best-of-N design loosens the acceptance gate and retains the top-N case studies by fix_rate. Table 8 and Figure 8 report RF test accuracy. The key observation is a test-ordering reversal on GS: unlimited wins at test time (78%) while best-of-3 led during training (89.3%), illustrating the risk of using train-accuracy metrics to select the case study count. All other differences are within noise.

Table 7: Phase 1 PK character limit ablation—RF test accuracy on gpt-4.1-mini (single-run).

Task	3 K	6 K	12 K	Unlimited
GS	**78.0%**	75.0%	57.0%	75.0%
FF	**98.0%**	97.0%	97.0%	**98.0%**
Snarks	**97.2%**	**97.2%**	**97.2%**	95.8%
DQ	70.0%	80.0%	**85.0%**	78.0%

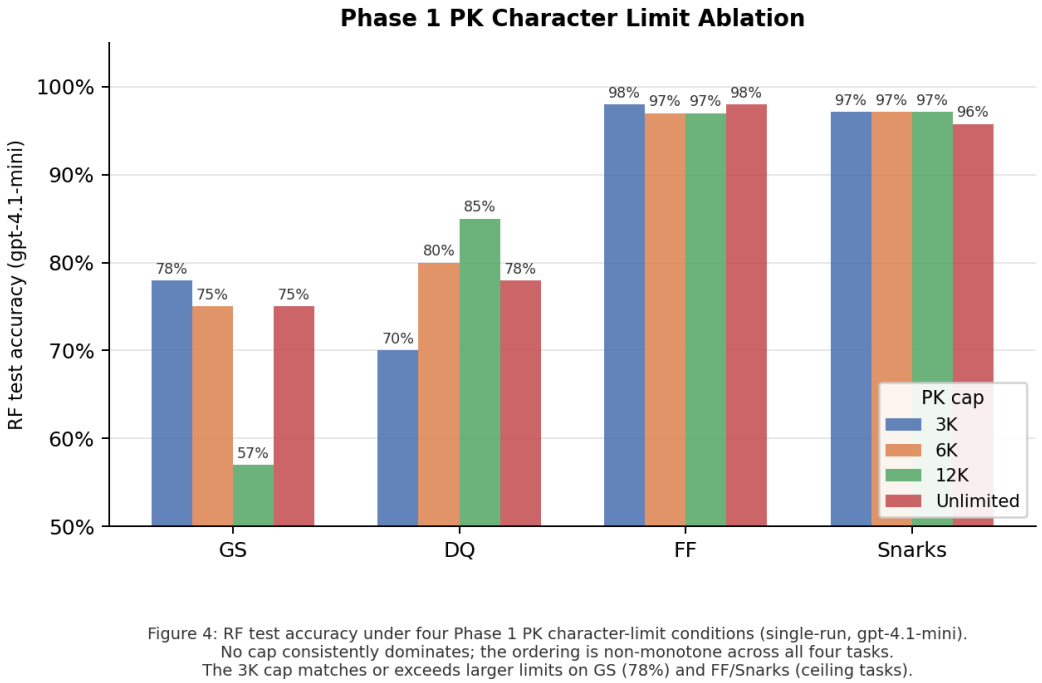

Figure 7: Phase 1 PK character limit ablation—RF test accuracy (single-run). The ordering is non-monotone; no cap consistently outperforms.

B Causal Judgement: Cross-Seed Case Study Inventory

The main text (Section 6.1) shows the single CJ case study from v3 seed 1, which closely paraphrases a specific training item's oracle reasoning (Listing 2). Table 9 lists all case studies generated across seeds 1–3 of the standard oracle-enabled v3 pipeline. The key observation is that seeds 2 and 3 do *not* exhibit the same oracle-derived specificity as seed 1: they produce structurally formatted ACTIVATE IF case studies rather than free-form narratives, and none directly paraphrase a training item. Nevertheless, *all three seeds produce case studies that hurt evaluation accuracy* (v3 full CJ non-train mean = 64.9–70.5% vs CS-ICL CJ = 72.4%), which implicates the *heterogeneous failure distribution* co-explanation: even structurally general case studies cannot cover 35 training failures spanning 6+ causal sub-types. Notably, seeds 2 and 3 converge on joint-AND causation and overdetermination patterns, missing norm violations, timing effects, intentionality judgements, and prevention chains entirely.

C Seed Variance Details

Table 10 gives the full per-seed numbers behind Figure 9.

The per-seed GS distribution for Llama under v3 (51%, 56%, 71%) shows that v3's vertex-counting rules are applied inconsistently—seed 3 Llama essentially solves GS while seeds 1–2 do not, indicating sensitivity to backend-routing variation at the token level. EA's structurally cleaner prior

Table 8: Phase 2 case study count ablation—RF test accuracy on gpt-4.1-mini (single-run). "—" indicates Snarks had no pool excess under the unlimited condition.

Task	Unlimited	Best-of-1	Best-of-3	Best CS (pool)
GS	**78.0%**	69.0%	74.0%	pool = 4, best = 100%
FF	**97.0%**	**97.0%**	**97.0%**	pool = 5, best = 50%
Snarks	**97.2%**	**97.2%**	**97.2%**	—
DQ	**88.0%**	**88.0%**	79.0%	pool = 4, best = 57%

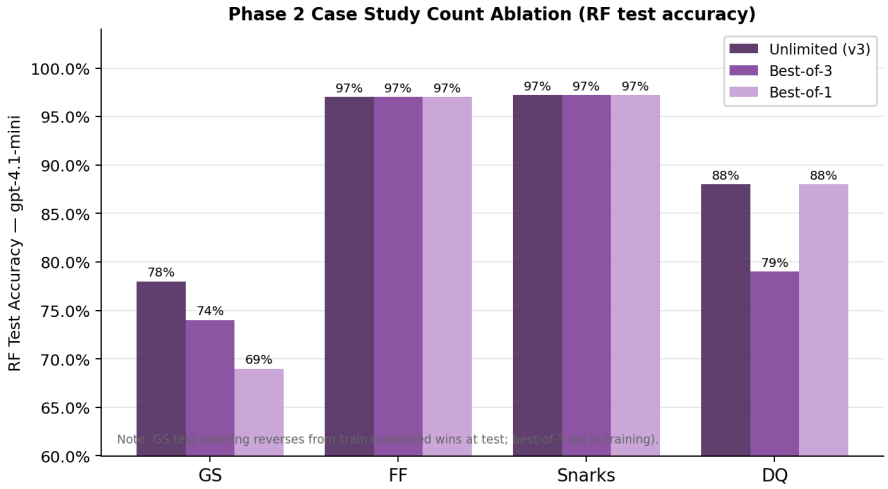

Figure 8: Phase 2 case study count ablation—RF test accuracy (single-run). GS test ordering reverses from train: unlimited wins. DQ shows best-of-3 performing worst. FF and Snarks are at ceiling across all conditions.

knowledge collapses this to a 3 pp range (68–72%), confirming a qualitative change in robustness rather than a mean-only shift. For CJ, the E3 improvement direction is consistent across all seeds and all models: every (model, seed) pair shows E3 $\geq$ v3, making the oracle-contamination finding the most seed-robust result in the evaluation.

Table 9: CJ case studies generated across all three v3 pipeline seeds. Seed 1: 1 oracle-contaminated case study (free-form narrative, garden fertilizer paraphrase). Seeds 2–3: multiple structurally formatted case studies addressing joint-causation and redundancy patterns. All three seeds produce case studies that reduce transfer accuracy relative to CS-ICL, implicating both oracle contamination (seed 1) and failure-distribution heterogeneity (seeds 2–3) as contributing factors.

Seed	**Case study title / pattern**	**Observation**
1	*Joint causes (logical AND): garden fertilizers*	Oracle-contaminated: free-form narrative paraphrasing one training item's gold chain-of-thought; encodes one idiosyncratic joint-AND scenario.
2	*Overlapping Contributors with Non-Intersecting Effects* (joint-AND causation)	Three structured ACTIVATE IF case studies, all addressing the joint-AND / combined-harm pattern from different angles. No oracle paraphrase; conditions are abstract. Covers only one causal sub-type; misses norm violations, timing effects, and intentionality judgements.
2	*Joint Necessary Actions Without Individual Effect*	
2	*Joint Harm Only from Combined Actions*	
3	*Redundant Contributors in Joint Outcomes* (overdetermination)	Four structured case studies spanning overdetermination, maintaining a sufficient condition, and compliance with instructions. More diverse than seed 2, but still omits norm violations, prevention chains, and intentionality subtypes. Two redundancy case studies are near-duplicates, paralleling the FF redundancy noted in Section 6.1—a recurring weakness of greedy partition search.
3	*Overdetermination with Redundant Contributors*	
3	*Causation by Maintaining a Sufficient Condition*	
3	*Causation by Complying with Essential Instructions*	

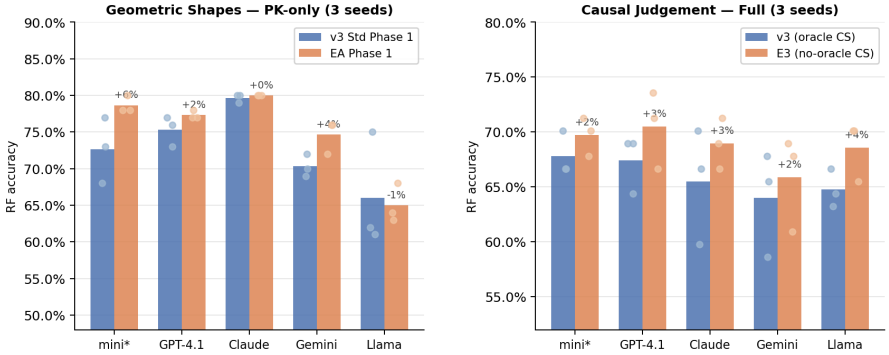

Figure 6: Per-seed (dots) and 3-seed mean (bars) RF accuracy for the two key ablation comparisons.
Left: EA Phase 1 raises GS PK-only by +2–6 pp across all models; EA results are more stable (Llama σ=4.7% vs v3 σ=10.4%).
Right: Removing oracle access (E3) improves CJ full accuracy for all five models (+1.9–3.8 pp), consistent with oracle contamination as a contributing factor; heterogeneous failure distribution is a plausible co-contributor (see Appendix B).

Figure 9: Per-seed (dots) and 3-seed mean (bars) RF accuracy for two key ablation pairs. *Left*: EA Phase 1 raises GS PK-only accuracy by $+2$ to $+6$ pp across models; EA seed-to-seed variance is substantially lower (Llama $\sigma = 4.7\%$ vs. 10.4%). *Right*: Removing oracle access (E3) improves CJ full accuracy for all five models by $+1.9$ to $+3.8$ pp, consistent with oracle contamination as a contributing factor; heterogeneous failure distribution is a plausible co-contributor (Section 6.2). Percentage labels show Δ (comparison minus v3 baseline).

Table 10: Per-seed RF accuracy for the two key comparisons (GS PK-only; CJ full). All values are test-set RF accuracy (%). v3: standard Phase 1. EA: evolutionary Phase 1. E3: oracle-free (both phases).

Task / Condition	**Model**	**S1**	**S2**	**S3**	**Mean**	σ
GS PK-only (v3)	mini	71	72	75	72.7	2.1
	GPT-4.1	80	83	82	81.7	1.5
	Claude	84	85	90	86.3	3.2
	Gemini	77	79	81	79.0	2.0
	Llama	51	56	71	59.3	10.4
GS PK-only (EA)	mini	79	79	78	78.7	0.6
	GPT-4.1	85	86	86	85.7	0.6
	Claude	89	88	91	89.3	1.5
	Gemini	82	82	84	82.7	1.2
	Llama	68	72	72	70.7	2.3
CJ full (v3)	mini	64	72	68	68.0	4.0
	GPT-4.1	60	71	71	67.3	6.4
	Claude	66	65	65	65.3	0.6
	Gemini	62	64	66	64.0	2.0
	Llama	61	65	68	64.7	3.5
CJ full (E3, no oracle)	mini	69	71	69	69.7	1.2
	GPT-4.1	68	71	72	70.3	2.1
	Claude	69	69	69	69.0	0.0
	Gemini	65	66	66	65.7	0.6
	Llama	66	70	70	68.7	2.3